Skylark BI Agent — Decision Log

1. Key Assumptions

- Pipeline value (Deals): sum of deal value excluding deals marked `Dead` or `Won` — these are the confirmed closed-status values found in the real `Deal Status` column.
- Active work orders: any `Execution Status` other than `Completed` counts as active/in-progress (Not Started, Ongoing, Partial Completed, etc.), matching how a founder would think about "what's still moving."
- Work Orders value field: I used "Amount Receivable (Masked)" as the default revenue/value column, chosen over ~7 other amount/billing columns in the sheet because it best matches generic "value" or "revenue" questions. A production system would let the user specify which billing stage they mean.
- Time filters ("this quarter," "this year") resolve against the real current date, not the dataset's date range. Since the sample data is historical, exact-period queries can legitimately return zero rows — the agent falls back to reporting the most recent period that does have data rather than failing silently, and states this fallback explicitly as a caveat.
- Metric routing is keyword-based on the free-text metric the LLM extracts (count / average / weighted / sum), not a fixed enum. This was a deliberate simplification given the 6-hour window.
- Single-board scope per query: each question resolves to one board (whichever the intent extractor identifies as primary). Cross-board comparison queries (e.g. "compare Deals pipeline to Work Orders billing side-by-side") are not supported — documented as a known limitation rather than silently guessed at.

2. Trade-offs

- **Groq-hosted Llama instead of local Llama 3.2**: I originally planned to use Llama 3.2 locally (I'd used it in a prior project), but a locally-hosted model isn't viable for a hosted, testable-without-local-setup prototype — there's no server to run it on in a Streamlit Cloud deployment. I switched to Groq's hosted inference API instead, which keeps the same open-weight Llama family but removes the local-hosting dependency entirely. Trade-off: adds an external API dependency and a small amount of latency per query, in exchange for actually being deployable.
- **Fuzzy matching for values vs. columns**: The fuzzy resolver was originally built to match free-text terms against categorical *values* in the data (e.g. "energy" → "Energy" in a Sector column), which works well for filters like sector or status. It initially had no way to resolve terms against *column names*, so a query like "missing start dates" incorrectly matched against the wrong column. I extended it with a second resolver specifically for column-name matching. This is documented in more detail as a fixed bug, but the underlying trade-off — treating value-resolution and column-resolution as separate

problems rather than one general fuzzy matcher — was a deliberate simplification to keep the initial build scoped, at the cost of an extra debugging pass.
- **Streamlit over a full custom front-end**: Given the 6-hour timeline, I chose Streamlit's chat interface over building a separate frontend/backend stack. This meant giving up custom UI control (styling, multi-panel layouts) in exchange for a working conversational interface, session state, and one-command deployment, all in far less time than an app would need.
- **Read-only, no caching layer**: every query hits monday.com live via the GraphQL API, per the assignment's requirement not to hardcode CSV data. I didn't add a caching layer (e.g. refresh every N minutes), so every query pays the full API round-trip cost. This was the right trade for correctness within the time available, but a production system handling frequent queries would benefit from a short-lived cache.

3. What I'd do differently with more time

- Add real cross-board query support (e.g. "which sectors are strongest in both pipeline and delivery") instead of routing every question to a single board.
- Add a lightweight caching layer on the monday.com pulls to reduce latency and API load for repeated queries within a session.
- Build automated tests around the query planner's filter/metric logic — right now correctness was verified by hand-running sample queries, which is fragile as the schema grows.
- Improve the fuzzy matcher's confidence threshold tuning with a labeled set of real founder-style questions, rather than a single fixed cosine-similarity cutoff.

4. How I interpreted "leadership updates"

I interpreted this as: the same underlying query engine, but a different output format tuned for pasting into a status update rather than reading conversationally. In the app, a sidebar toggle ("Leadership update mode") switches the answer-synthesis prompt from a 3–5 sentence conversational answer into 2–3 short bullet points — still fully grounded in the same computed data and caveats, just formatted for reuse. I chose not to build a separate "generate report" feature, since the assignment scope already required a lot within 6 hours, and a formatting toggle achieves the spirit of "help prepare data for leadership updates" without adding a new subsystem.

5. monday.com Setup

I created one workspace with two boards: "Deals" and "Work Orders." Each CSV was imported directly into its board, and I manually set the data type for each column (text, status/dropdown, date, number). I deliberately did not clean, impute, or alter any missing or messy values during import — nulls, inconsistent casing, and stray formatting were left exactly as in the source data, so the agent's data-resilience logic (null handling, fuzzy matching, caveats) is being tested against real messy data rather than data I'd already cleaned by hand.